

E2E-Automation-Optimized 项目优化方案

背景与问题

E2E-Automation-Optimized 是一个企业级的本地多 Agent 测试自动化系统，旨在通过多智能体协作实现端到端（E2E）测试流程的自主执行¹²。该系统采用 Kiro CLI 作为 Agent 调度工具，结合 Beads (bd) 进行任务和依赖管理，并使用标准化的 Agent SOP 文档和 Skills 来执行测试步骤。经过多阶段的迭代完善（知识提取、工作流标准化、任务管理集成、单Agent优化、多Agent系统等），项目已经具备了一定规模³⁴。

然而，当前项目仍存在一些亟待解决的问题：

- **Agent 执行逻辑不够稳定，协作链条不清晰：**多个 Agent 协同工作时，任务衔接和状态传递不明确，偶尔出现流程中断或 Agent 行为异常，表明调度逻辑有待加强。
- **SOP 文档过多且维护困难，结构化程度弱：**目前系统包含 7 个 SOP 文件涵盖不同场景⁵，内容庞杂且部分流程重复，缺乏模块化组织，增加了编写和更新难度。
- **Agent 调用链不明确，缺少自动衔接机制：**各 Agent 的职责虽在定义上区分清楚，但缺乏自动链式触发机制。例如，一个 Agent 发现问题后如何通知下一个 Agent 继续处理，之前并未严格自动化，导致需要人工介入或约定流程才能衔接。
- **项目目录结构和文档较繁杂，缺乏模块化分层：**现有目录中配置、脚本、文档交织，一些关键配置隐藏在深层路径（如 `.kiro` 目录）下，不利于理解和扩展。新增功能（如 CI/CD 集成、性能测试）缺少清晰的放置位置。

上述问题限制了系统在更复杂场景下的扩展和维护。为此，我们在保持核心工具（Kiro CLI、Beads、Agent SOP 等）不变的前提下，对系统的架构和组织方式进行优化调整，提升多 Agent 协作的**稳定性、清晰性和可扩展性**。

优化目标

为解决上述问题，优化工作的主要目标如下：

1. **优化工具使用方式与组织结构：**在不更换 Kiro CLI 调度、Beads 任务管理、Agent SOP 这些核心组件的前提下，调整它们的使用方法和配置组织，使系统更易于理解和维护。
2. **改进多 Agent 调度方案：**设计更加稳定清晰的多 Agent 协作机制。重点优化 `orchestrator.py` 主调度脚本的结构和执行策略，引入**并发处理能力**并增强**错误恢复机制**，保证各 Agent 按预期顺序协同工作。
3. **重构 SOP 文档结构与编写方式：**对现有 SOP 进行归类分层，提炼通用步骤，减少重复。采用模板化的方法撰写 SOP，使其**更易维护、易扩展**，支持根据不同角色/功能快速生成标准流程文档。
4. **调整项目目录结构：**对项目文件夹布局进行重组，按照功能模块清晰分层，为将来新增功能（如持续集成/部署，性能测试模块等）预留位置，保证扩展时不影响现有结构。
5. **输出完善的架构与规范文档：**本文档将以 Markdown 格式给出完整的优化方案说明，涵盖优化后的架构设计、目录结构、核心脚本设计建议、SOP 模板格式、Agent 调度规范、Beads 使用建议等内容，供团队参考实施。

以下将按照上述目标逐项阐述具体的优化方案。

优化后的系统架构

优化后，E2E-Automation-Optimized 系统采用**分层架构**，将不同职责的模块清晰划分为四层，如下表所示：

层次	主要组件	作用描述
知识库层 (Knowledge Base)	关键规则、检查点文档，通用问题解决方案文档等	提供测试领域的经验规则和核对清单，指导 Agent 决策 ⁶ ⁷ （例如22条关键规则、4个验证检查点等）。
工作流程层 (Workflow)	Agent SOP 文档（源文件），Skills 文件（供AI调用的格式）	定义标准化的工作流程和步骤。SOP 以 RFC 2119 格式编写，经过转换生成 Anthropic 格式的 skill 提供给 Agent 调用 ⁵ ⁸ 。这一层确保每个 Agent 按既定流程执行，并支持 Skills 链式调用 等自动衔接机制 ⁸ 。
任务管理层 (Task Management)	Beads 任务跟踪系统（SQLite 数据库 + JSONL 文件）、依赖关系机制、命名规范配置	提供跨会话的任务持久化和依赖管理。任务以 issue 的形式存储于 Beads（本地 SQLite 并定期同步到 JSONL 以便 Git 版本控制） ⁹ 。支持任务 阻塞/解阻塞 表示依赖关系 ¹⁰ 。定义 任务命名规范 以辅助 Master 调度正确路由任务 ¹¹ 。
执行层 (Execution)	单 Agent 执行模式（Kiro CLI 交互），多 Agent 模式（Python Orchestrator + 多个 Worker Agent）	负责具体任务的执行与协作。单Agent模式下由人工驱动单一 Agent 完成所有步骤；多Agent模式下由 Python 实现的 Master Orchestrator 协调多个专职 Worker Agent 协同完成复杂任务，实现 事件驱动的自动协调 ¹² ¹³ 。

上述架构将知识、流程、任务、执行解耦分层，各层之间通过清晰接口互动：知识库为工作流程提供规则依据，工作流程指导执行层的 Agent 操作，任务管理层通过事件（任务就绪/阻塞）将执行层各Agent串联起来，形成闭环。

多Agent协作模式： 在执行层，优化后的系统更强调**专业化分工与事件驱动调度**。即由一个 **Master Orchestrator（主调度器）** 统一监听任务队列，并根据任务类型将工作分配给不同专长的 Worker Agent。以 E2E测试为例，系统划分出三个主要的 Worker Agent：

- **Test Creator** – 负责测试用例的生成与初步执行（通用测试创建）。
- **Bug Fixer** – 负责定位并修复测试失败中暴露的应用缺陷（例如专门的前端 Bug 修复）。
- **Test Validator** – 负责在测试通过后进行结果验证（对比实际结果与期望参考，确保没有遗漏问题）。

Master Orchestrator 和各 Worker Agent 共同构成 **事件驱动的多Agent协同机制**：测试任务开始→创建测试→执行测试→发现Bug则产生新任务→修复Bug→继续测试→测试通过则验证→完成。这种架构极大提高了复杂测试场景下的自动化处理能力，并确保每个Agent各司其职、按序衔接 ¹⁴ ¹⁵。

下面我们将详细介绍多Agent调度方案的改进细节，以及相关核心模块的设计。

多 Agent 调度方案改进

调度流程概述

优化后的多Agent调度采用**主从协调**模型：由 Master Orchestrator 作为主控循环，不断从 Beads 中获取“就绪（ready）”状态的任务，根据任务类型选择合适的 Worker Agent 执行。当任务因为等待其他任务完成而处于“阻塞（blocked）”状态时，Orchestrator 会跳过执行并定期检查其依赖是否解除，从而实现事件驱动的调度。整个流程是一个持续运行的循环，直至所有任务完成为止 16。

下面以一个完整的 E2E 测试场景为例，说明优化后的多Agent协作流程：

步骤	动作与任务变化	负责实体
1	创建新的测试任务（如 TC06）并加入 Beads 任务列表，状态为 ready。	用户或触发源
2	Master Orchestrator 从 Beads 获取到就绪任务列表，选取最高优先级的 TC06 测试任务 17 并分配给 Test Creator Agent 执行。	主调度器
3	Test Creator Agent 根据 SOP 生成测试脚本并尝试执行测试。若在执行过程中发现应用存在 bug（例如缺失某项功能或断言失败），则调用 Beads 创建一个新的 Bug 任务，并将当前测试任务标记为被该 Bug 阻塞 18。随后 Test Creator 提前结束，将 bug 情况报告给 Orchestrator。	Test Creator Agent
4	Master Orchestrator 发现 TC06任务已被阻塞，同时检测到新的 Bug 任务就绪。根据任务类型，Orchestrator 将此 Bug 任务指派给 Bug Fixer Agent 处理 19 20。	主调度器
5	Bug Fixer Agent 获取 Bug 描述（自动从 Beads 中读取任务详情 21），加载相应的修复流程 SOP，执行代码修改来修复缺陷。修复完成后，Bug Fixer 验证问题是否解决（例如运行相关单元测试或根据SOP提示进行确认 22），然后将 Bug 任务状态标记为关闭（已修复） 23。	Bug Fixer Agent
6	Bug 任务完成后，先前被阻塞的测试任务 TC06 自动解除阻塞（Beads 检测到其依赖的 Bug 任务已关闭），重新变为就绪状态 24。Master Orchestrator 在下一个循环中再次将 TC06 分配给 Test Creator（或专门的测试执行Agent）继续执行。	主调度器
7	Test Creator Agent 重新运行测试用例。由于缺陷已修复，此次测试通过所有检查。Test Creator 按 SOP 将测试任务标记为完成（状态 closed）并输出成功结果 25。	Test Creator Agent
8	（可选）测试通过后，为进一步验证产品行为的正确性，系统可以创建一个“验证任务”（如 Validate TC06），用于对测试结果进行独立校验（比如截图对比、数据完整性检查）。可以由 Test Creator Agent 自动创建此任务，或由 Orchestrator 在测试任务完成时自动生成。	Test Creator Agent / 主调度器
9	Master Orchestrator 将 验证任务 分配给 Test Validator Agent 处理。	主调度器
10	Test Validator Agent 加载参考验证 SOP，读取预先保存的期望结果和测试实际输出，进行详细对比分析（如截图像素比对、日志检查等），判断测试结果是否真实符合预期规格 26 27。验证完成后关闭验证任务，输出报告 28 29。	Test Validator Agent

步骤	动作与任务变化	负责实体
11	所有相关任务（测试任务及其派生的 Bug、验证任务）均完成且无阻塞，Master Orchestrator 检查到 无就绪或阻塞任务 ，宣布流程结束并打印总结统计 ¹⁶ ³⁰ 。整个 E2E 测试循环圆满完成。	主调度器

通过以上链式任务驱动，系统实现了多Agent的自动协作：各 Agent 基于任务类型被动态唤起，各尽其职地完成测试、定位问题、修复代码、验证结果等工作，并通过任务的创建/阻塞/完成状态来传递信息和同步流程。**协作链条**清晰且高度自动化，极大减少了人工干预。

Master Orchestrator 设计优化

Master Orchestrator（主调度器）是整个多Agent模式的中枢，其实现直接影响系统稳定性和效率。本次优化对 `master_orchestrator.py` 脚本做出多项改进：

1. 模块化设计：将 Orchestrator 的主要功能划分为独立的方法和类，增强可读性和可维护性。例如，引入 `BeadsClient` 类封装 Beads CLI 的调用（如获取任务、更新状态、同步等）³¹ ³²；引入 `WorkerAgent` 类封装对单个 Kiro CLI Agent 的调用³³ ³⁴，包括构建 prompt 和执行命令的逻辑，使主循环代码更简洁。

```
# 简化的 Beads 客户端示例
class BeadsClient:
    def ready(self) -> List[Dict]:
        result = subprocess.run(["bd", "ready", "--json"], capture_output=True, text=True)
        return json.loads(result.stdout) if result.returncode == 0 else []

    def blocked(self) -> List[Dict]:
        result = subprocess.run(["bd", "blocked", "--json"], capture_output=True, text=True)
        return json.loads(result.stdout) if result.returncode == 0 else []
```

上述示例展示了 BeadsClient 中对 `bd ready` 和 `bd blocked` 的封装，使主调度器可以方便地获取任务列表而无需处理底层系统调用细节。

2. 明确的任务分配策略：Orchestrator 采用**任务类型 + 标题关键字**的策略来选择执行任务的 Agent，实现**稳定的路由**。具体规则定义在 `select_worker()` 方法中³⁵ ³⁶：

- 若任务类型为 `task` 且标题包含“TC”（代表测试用例ID），则分配给 **Test Creator**（测试创建Agent）³⁷。
- 若任务类型为 `bug` 或标题包含“bug”/“fix”，则分配给 **Bug Fixer**（缺陷修复Agent）³⁸。
- 若标题包含“validate”或“validation”，则分配给 **Test Validator**（结果验证Agent）³⁹。
- 否则默认分配给 Test Creator 处理⁴⁰。

这一策略与 **任务命名规范** 配合，使每个任务由正确的Agent处理，避免职责混淆（见下一节详细说明命名规范）。如⁴¹所示，当任务标题符合相应关键词时，Orchestrator 将正确选择对应的 Worker⁴² ⁴³。这保证了多Agent协作链的**一致性和可预期性**。

3. 并发处理能力：为提高效率，优化方案规划引入**并发执行**支持。当 Beads 有多个相互独立的任务处于就绪状态时，Orchestrator 可并行启动多个 Agent 处理不同任务。实现上，可以采用 Python 的多线程或异步IO来启动多个 `WorkerAgent.execute()` 子过程，同时跟踪它们的完成状态。

并发设计建议：使用线程池管理 Worker 调用，例如：检查当前有多少任务就绪，若超过1个且系统资源允许，则创建新线程调用相应的 WorkerAgent 去执行任务。需要注意对共享资源（如代码仓库）的互斥访问：例如避免多个 Agent 同时修改同一代码模块。可通过在 Bug Fixer 执行期间暂缓新的 Test 执行，或采用任务锁机制，确保并发运行不会互相干扰。

```
# 并发执行的伪代码示例
from concurrent.futures import ThreadPoolExecutor

with ThreadPoolExecutor(max_workers=3) as executor:
    for task in beads.ready():
        worker = select_worker(task)
        executor.submit(worker.execute, task)
# 等待所有并发任务完成（或设置超时）
```

通过引入并发，系统在中期规划中将支持同时处理多项任务（例如同时创建多个测试用例），提高整体吞吐量⁴⁴。当前版本暂未启用多线程（未来计划项）⁴⁴，本方案为并行扩展提供了设计基础。

4. 健壮的错误恢复：Orchestrator 主循环对可能出现的异常情况做了完善处理，保证系统稳定运行不断线：

- **Agent 超时：**如果某个 WorkerAgent 执行超出设定时长（如30分钟），则 Orchestrator 捕获 `TimeoutExpired` 异常，将对应任务标记为出错状态并计入失败次数，然后不中断主循环，继续处理其他任务⁴⁵。这样长时间挂起的任务不会阻塞整个系统。
- **Agent 异常：**如果调用过程中抛出其他异常，Orchestrator 捕获并记录错误详情，同样将任务状态更新为错误⁴⁶。通过统计失败计数，管理员可以在日志中发现反常情况。
- **任务阻塞监测：**当一轮循环中没有任何任务可执行时，Orchestrator 会检查是否存在被阻塞的任务。如果存在，则进入等待休眠（如每次60秒）并累计等待计数⁴⁷⁴⁸。如果连续多轮等待仍未有任务解除阻塞，Orchestrator 会发出警告提示可能出现**死锁**或长时间无响应⁴⁹⁵⁰。管理员可据此介入检查异常阻塞原因。相反，如果既无就绪任务也无阻塞任务，则说明所有任务完成，Orchestrator 安全退出循环¹⁶。这种机制避免了调度器陷入无限空转或挂起。
- **自动恢复执行：**当某任务被阻塞时（通常由于等待Bug修复），Orchestrator并不终止该任务，而是在关联的Bug任务完成后自动将其重新纳入就绪队列继续执行，从而**自动恢复流程**¹⁵。例如上例中 TC06 测试任务在Bug修复完成后被解阻，Master下一轮又会安排它运行，无需人工干预。这一特性保证了测试流程可持续推进，符合无人值守运行需求。

经过上述优化，新的 Master Orchestrator 结构更加清晰，执行逻辑健壮可靠。其核心循环如同一个守护进程，不断监控任务状态变化，通过**任务事件**驱动多Agent协作，在保证顺序合理性的同时提高了效率。一旦某环节出现问题（超时、异常），系统也能适当处理而不崩溃，为大规模长时间运行提供了保障。

Agent 任务命名规范与自动衔接

为确保 Master Orchestrator 正确路由任务给相应的 Agent，并方便 Agent 之间创建后续任务，本方案强化了**任务命名规范**和**SOP 内链式指引**：

1. 任务命名规范： 项目制定了明确的任务命名约定¹¹，要求在使用 `bd create` 创建任务时，按任务类型和目的选择合适的类型标识和标题前缀^{51 52}：

- **测试任务**（新测试用例开发）：使用类型 `task`，标题必须包含测试用例标识“TC##”字样⁵³。例如：`TC06: Update Location OFS Settings`⁵⁴。这确保 Orchestrator 能识别为测试创建任务并指派 Test Creator。^{53 54}
- **缺陷任务**（Bug 修复）：使用类型 `bug`，标题应以“Bug:”或“Fix:”开头，加简要描述^{52 55}。例如：`Bug: Missing i18n string for SMG_YAMATO`⁵⁶ 或 `Fix: i18n error in settings page`。这样 Orchestrator 将其视为 Bug 任务并指派给 Bug Fixer。¹⁹
- **验证任务**（结果核验）：可使用类型 `task` 或 `chore`，但标题中应包含“validate”或“validation”关键词^{57 58}。如：`Validate TC06 results against reference` 或 `TC06 Validation`⁵⁹。Orchestrator 将据此指派给 Test Validator 处理。⁶⁰
- **其他任务**：非以上形式的任务默认视为一般测试任务，由 Test Creator 处理。如需新增任务类型，可扩展命名规则并在 Orchestrator 中更新对应逻辑。

通过上述规范，开发人员或 Agent 在创建后续任务时有据可循，避免出现含糊不清的任务描述。例如，不应创建标题模糊的任务如“Something wrong with page”或“Check TC06”，否则 Master 可能无法正确路由^{56 61}。实际案例显示，使用正确的命名能避免任务分配错误：如 Test Creator 发现 Bug 时，应创建 `Bug: ...` 类型任务⁶²，而非误用 `-t task` 导致被错误路由回 Test Creator⁶³。

为了强制执行这一规范，我们在多处进行了约束：

- **Agent 提示与 SOP 约定：** 在 Test Creator 的 Agent prompt 和主要 SOP (`e2e-test-automation.sop.md`) 中明确规定：“当你发现 Bug 时，必须按照 `Bug: ...` 格式创建任务”^{64 65}。同样，对于需要验证的情况，也指示使用“Validate”字样创建验证任务^{66 67}。这些指引确保 AI Agent 自行产生后续任务时符合规范，从源头杜绝不当命名。
- **Beads 命名检查：** Beads 的命名规范文档 (`BEADS-NAMING-CONVENTION.md`) 作为资源提供给 Agent 参考⁶⁸。Agent 可被动查阅其中的规则说明，从而了解如何正确命名任务以协同工作。¹¹

通过严格的命名规范，Master-Orchestrator-Worker 的链条被固化：每个任务的产生、路由、执行都有迹可循。这种显式协议大大提高了多 Agent 协同的可靠性，使**自动链式衔接**成为可能：Agent A 发现的问题，通过标准任务形式交由 Agent B 解决，Agent B 完成后再引发 Agent A 或 C 跟进，无需人工翻译或干预任务流转。

2. SOP 内的链式衔接指引： 除了命名约定，本次优化还调整了 SOP 编写，使其在适当步骤直接指导 Agent 产生日志或后续任务，串联起跨 Agent 流程。例如：

- 在测试流程 SOP 的 Bug 处理步骤中，加入**创建 Bug 任务并阻塞当前任务**的操作说明。正如 Test Creator Agent 的执行 Prompt 中第 5 步所示：¹⁸

```
“If you discover bugs during execution:  
- bd create "Bug: <description>" -t bug -p 0 --json  
- bd dep add {task_id} <new-bug-id> --type blocks --json  
- bd update {task_id} --status blocked --json  
- bd sync  
- Return immediately with message about the bug”
```

这一系列指令让 Test Creator 在发现问题时，**立即产生日志并生成Bug任务**，同时通过依赖设置阻塞住当前测试任务，确保流程有序暂停 18 24。这样 Master 检测到Bug任务就绪后，会先处理Bug。

- 在测试完成后的 SOP 中，建议新增一步用于**触发结果验证**。例如，在 `e2e-test-automation.sop.md` 结尾处添加：“测试通过后：bd create 'Validate <TC编号>' -t task-p1”，引导 Agent 自动创建验证任务 69 70。这样当测试任务完成时，验证任务已经准备好进入队列，由 Orchestrator 分派给 Validator 执行。

通过在 SOP 中显式加入跨Agent衔接步骤，各 Agent 的输出变成下一个 Agent 的输入。这实现了一种**隐式的 Agent 调用链**：不直接让Agent调用另一个Agent，而是通过任务事件驱动 Master 来调用，既保持了每个Agent的独立性，又达到了链式自动执行的效果。

Agent SOP 结构优化与模板

为解决当前 SOP 文档数量多、结构重复且维护困难的问题，本方案对 SOP 体系进行了重构，主要策略包括**分层归类、模板约束和内容复用**。

1. 分层分模块组织SOP：根据Agent角色和功能，将现有流程文档划分为几类，便于聚合管理：

- 测试设计与执行类：**包含测试创建主流程 SOP（如 `e2e-test-automation.sop.md`）以及相关子流程 SOP（如从规范文档生成测试的 `test-design-from-quip.sop.md`，测试脚本修正的 `e2e-test-fix.sop.md` 等）。这些 SOP 服务于 **Test Creator** Agent，涵盖从获取测试需求到编写、运行测试用例的整个生命周期。
- 缺陷分析与修复类：**包含针对代码问题的 SOP，如前端缺陷修复流程（`frontend-code-fix.sop.md`）以及通用问题排查（`common-issues-troubleshooting.sop.md`）等。这类SOP主要供 **Bug Fixer** Agent 使用，指导其定位问题、应用规则修复并验证修复有效。
- 结果验证类：**专门用于测试结果对比验证的 SOP，例如 `reference-material-validation.sop.md`（参考资料验证）。这类 SOP 由 **Test Validator** Agent 使用，指导其如何收集比对测试输出和期望结果，包括应用检查点、规则避免误判等。 71 27
- 辅助/高级功能类：**如 `feature-e2e-autopilot.sop.md`（自动驾驶测试功能）等，提供一些高级用例的流程。这部分可以独立成模块，按需加载，不影响主流程。

通过以上分层，相关的 SOP 文档归组放置，既减少了单个Agent需要加载的内容，也方便了局部修改和扩展。例如，在增加一个新的 Worker Agent（如性能测试Agent）时，可以在相应类别下新增一组 SOP 文档，而无需梳理整个体系的关联。

2. SOP 模板化编写：制定统一的 SOP 文档结构模板，确保所有 SOP 在格式和风格上保持一致，提高可读性并方便日后自动解析或转换为Skill。推荐的 SOP Markdown 模板如下：

SOP 名称（命令式标题）

概要 (Overview)

简要描述此 SOP 的目的和适用范围。

参数 (Parameters)

- ****param1****（必需/可选）：描述参数用途或约束。


```

- param2: ...

* (如无参数可省略本节) *

## 步骤 (Steps)
1. 步骤标题/目的: 描述该步骤要做什么。
   Constraints: 列出该步骤需要遵循的约束或注意事项 (使用RFC 2119关键词, 如 MUST/SHOULD) 。
2. 步骤2标题: ...
   Constraints: ...
3. ...

## 示例 (Examples)
给出一个或多个该流程的实例应用场景, 包含输入输出示例, 帮助理解如何执行。

```

各 SOP 作者在撰写时应遵循上述模板, 包括固定的章节顺序和格式。这带来以下好处:

- **格式一致**: 统一的标题、编号和用词风格 (如强调 MUST/SHOULD 等 ⁷² ⁷³) 使不同 SOP 看起来风格一致, 降低理解成本。
- **易于转换**: 模板格式便于用脚本解析, 将 SOP 源Markdown转换为不同用途的输出。例如可通过脚本抽取 “步骤” 部分, 自动生成 Anthropic Skill 的 YAML+Markdown 文件格式 ⁷⁴。事实上当前项目已采用Anthropic技能格式 (YAML frontmatter + Markdown正文) 来发布技能 ⁷⁵, 模板使这种转换更加机械化, 避免手工错误。
- **内容复用**: 常见的约束说明或检查项可以通过模板约定集中管理。例如所有 SOP 末尾都可以附加一个标准的 “完成条件” 列表, 列出任务完成所需满足的条件 (测试通过、结果验证、日志无误等)。当这些标准更新时, 可统一调整模板而各 SOP 自动应用。

3. 提炼通用片段避免重复: 分析现有 SOP, 提炼出多文档重复的内容, 迁移到公共资源或单独的 SOP 片段, 以引用或include方式使用:

- **规则/检查点**: 原先各 SOP 可能分别列出了关键规则或验证点, 现在将它们集中记录在知识库层的文件中 (如22条关键规则、4个检查点分别在 `E2E-CRITICAL-RULES.md` 和 `E2E-VALIDATION-CHECKLIST.md`), 各 Agent 通过资源引用获取 ⁷⁶ ⁷⁷。SOP 中不再重复列出规则细节, 而是在需要用到时提醒应用某条规则即可。这样如果规则更新, 只需改动一处文档即可同步所有流程的依据。
- **通用步骤序列**: 一些典型的步骤序列可作为独立的小流程重用。例如 “Bug 修复验证步骤” (编译代码->重跑测试->比对日志等) 可以写成一个通用子SOP, 由不同 bug fix SOP 引用; 又如 “结果比对通用流程” (截图+HTML双重检查) 在不同验证SOP间大同小异, 也可抽取成公共说明段落 ⁷⁸ ⁷⁹。
- **工具使用方法**: 针对经常使用的工具命令, 如 `bd` 系列操作 (`bd show/update/close`等) 或测试框架命令, 可以在附录文档中详细说明使用方法, 在 SOP 中简单引用链接。这样既减少正文冗长说明, 又确保工具使用的一致性。

通过模块化和复用, SOP 文档从 “散装的脚本” 提升为 “结构化知识模块”。开发者在增加新 SOP 时, 可以基于模板快速创建初稿; 当修改某个通用流程时, 也能一次改动影响所有相关 SOP, 保证流程的一致和正确。

4. 版本管控和审阅: 所有 SOP 文档应纳入版本控制, 并建立审阅机制。因为 SOP 是给AI看的 “剧本”, 其准确性和逻辑性非常重要。在更新SOP时, 建议至少进行一次完整的Agent模拟运行测试, 确保新的SOP不会破坏既有功能。此外, 可考虑将关键SOP (主流程等) 打印出流程图或关键步骤列表, 方便人类校验其合理性。

经过上述优化，SOP 体系将更加清晰易懂。**标准化的格式**提供了全局一致性，而**模板化和模块化**使维护量大大降低，新功能的流程编写也更快捷——这为系统未来扩张（增加新Agent角色、新业务流程）奠定了良好的基础。

项目目录结构优化

优化后的项目目录充分体现分层架构理念，将文件组织按功能归类，并为未来扩展预留空间。下表列出了调整后的目录结构及说明：

E2E-Automation-Optimized/	# 项目根目录
├── master_orchestrator.py	# Python 主调度器脚本 80 81
├── start_orchestrator.sh	# Orchestrator 启动脚本 82 83
├── .beads/	# Beads 任务管理数据 84
│ ├── beads.db	# 本地SQLite数据库（任务存储） 85
│ └── issues.jsonl	# 任务日志(JSON Lines, 用于Git同步) 86
├── config/	# 配置和知识库层（项目内配置）
│ └── .kiro/	# Kiro CLI 配置目录
│ ├── agents/	# Agent 配置文件目录 87
│ ├── e2e-testing.json	# 单Agent模式配置（综合Agent）
│ ├── test-creator.json	# 多Agent模式 - 测试创建 Agent 88
│ ├── bug-fixer.json	# 多Agent模式 - 缺陷修复 Agent 89
│ └── test-validator.json	# 多Agent模式 - 结果验证 Agent 90
│ └── steering/	# 知识库文件目录 91
│ ├── E2E-CRITICAL-RULES.md	# 22条关键规则 76
│ ├── E2E-VALIDATION-CHECKLIST.md	# 4个验证检查点 92
│ ├── E2E-GLOBAL-RULES.md	# 通用全局规则
│ └── BEADS-NAMING-CONVENTION.md	# 任务命名规范 68
├── sops/ (or config/.kiro/sops/)	# Agent SOP 源文件目录 93
│ ├── e2e-test-automation.sop.md	# 主流程 SOP（测试创建/执行）
│ ├── frontend-code-fix.sop.md	# 前端缺陷修复 SOP
│ ├── e2e-test-fix.sop.md	# 测试用例修正 SOP
│ ├── common-issues-troubleshooting.sop.md	# 通用问题排查 SOP
│ ├── reference-material-validation.sop.md	# 结果验证 SOP
│ ├── test-design-from-quip.sop.md	# 从规范生成测试设计 SOP
│ └── feature-e2e-autopilot.sop.md	# E2E自动驾驶功能 SOP
├── skills/	# 转换后的 Skills 文件（供AI加载） 94
│ ├── e2e-test-automation/...	# 对应各 SOP 转换的 skill（7个项目相关） 95
│ ├── frontend-code-fix/...	
│ ├── ... (其它同名目录)	
│ ├── code-assist/...	# 内置Skill示例（如代码辅助） 96
│ ├── code-task-generator/...	# 内置Skill示例
│ └── ... (更多内置 Skills)	
├── docs/	# 文档资料目录 97
│ └── guides/testing/	# 测试指南

```

|   |   |—— COMMON-ISSUES-AND-SOLUTIONS.md # 常见问题与解决方案知识库 98 99
|   |   |—— E2E-LESSONS-TABLE.md # E2E经验教训表
|   |—— test-cases/ # 测试用例说明
|   |   |—— index.yaml
|   |   |—— TC06-UPDATE-LOCATION-OFS.md # 测试案例06说明
|   |—— BEADS-SETUP-COMPLETE.md # Beads 使用指南
|   |—— EVENT-DRIVEN-MULTI-AGENT-DESIGN.md# 多Agent架构设计 100
|   |—— MULTI-AGENT-QUICKSTART.md # 多Agent模式快速上手
|   |—— MULTI-AGENT-ARCHITECTURE.md # 多Agent架构方案对比
|   |—— SYSTEM-OPTIMIZATION.md # 系统优化方案（针对误判问题等） 101
|   |—— README.md # 项目总览说明
|
|—— tests/ # 测试脚本和结果目录
|   |—— e2e-tests/
|   |   |—— tests/ # 测试脚本文件
|   |   |—— fixtures/ # 测试数据夹具
|   |   |—— test-results/ # 测试输出结果（截图、日志等）
|
|—— (未来新增目录) # 预留的扩展功能模块
|   |—— ci_cd/ # CI/CD 集成脚本与配置（如流水线定义）
|   |—— performance/ # 性能测试相关Agent及脚本
|   |—— integrations/ # 外部系统集成（如Slack通知、Web UI监控）
|   |—— ...

```

说明：上述结构中带有 **(未来新增目录)** 的部分为本次建议新增或预留，用于后续扩展，不一定在当前项目中存在。

优化亮点分析：

- **层次清晰：** 根目录下区分了执行脚本、配置、工作流程(SOP/Skills)、文档、测试等模块，大大提高了可导航性。比如只看目录就能发现 `.beads` 存储任务数据，`config/.kiro` 下是Agent和知识库配置，`skills` 下是AI模型所需技能文件，互不混杂。
- **配置集中：** Kiro CLI 相关的 Agent 配置和 Steering 规则全部集中在 `config/.kiro/` 下，方便统一管理和版本控制（也便于和用户级别的 `~/.kiro` 区分开 102）。同理，所有SOP源文件集中在一个文件夹下，便于编写者查找修改。
- **文档集中且索引：** `docs/` 下按照用途细分指南 (guides)、案例 (test-cases)、架构设计、优化方案等文档，目录清晰易查。同时在 README 或索引文档中应列出各关键文档的简要说明 103，方便新人快速找到所需信息。
- **预留扩展位：** 目录结构预见性地为**CI/CD集成、性能测试、新Agent**等预留了位置。未来如果要添加性能测试Agent，只需在 `agents/` 添加配置，在 `sops/` 添加SOP，在 `skills/` 添加技能文件，并将性能测试相关脚本放入 `performance/` 目录，即可按现有模式扩展，不会破坏原有架构。此外，如果需要加入Slack通知或Web界面监控，也可以在 `integrations/` 目录中开发独立模块，通过Orchestrator钩子调用，避免散落各处。
- **测试与代码隔离：** 测试相关的代码/数据放在 `tests/` 下，与产品代码或Agent配置相隔离。这也为将来实现**持续集成** (CI) 提供便利——CI只需跑 `tests` 目录下的内容。同时测试结果存档在 `test-results/`，有助于Agent验证结果 (Validator Agent 会去此处比对截图日志) 以及问题追溯。

- **脚本标准化：** 提供统一的启动脚本如 `start_orchestrator.sh`，对环境依赖检查、启动参数配置等进行规范 ¹⁰⁴ ¹⁰⁵。这样使用者不必每次记复杂命令，只需运行脚本即可启动整个多Agent系统，提升可用性。

总之，新目录结构以**高内聚、低耦合**为目标，将相关内容归类在一起，同时使不同模块边界清晰。开发者阅读项目时，可以按层次逐步深入：先看 README 和架构文档了解整体，再看 `config/agents` 配置Agent细节，`sops/skills`了解流程脚本，最后深入 `orchestrator.py` 看运行机制。整个项目如同一本结构良好的手册，易读易扩展。

Beads 任务管理与依赖管理建议

Beads (bd) 作为本项目的任务和依赖管理工具，在多Agent系统中发挥了关键作用。优化方案充分利用了Beads 的特性，并提出一些使用建议以确保任务流转高效可靠：

1. 善用任务状态和优先级：

Beads 将任务分为不同状态（如 `ready`，`in_progress`，`blocked`，`closed`，`error` 等）和优先级（priority）队列。建议遵循以下实践：

- 创建任务时合理指定 **优先级**（`-p` 参数）。例如，新测试任务可赋予较高优先级1，而派生的Bug任务优先级为0 ¹⁰⁶ ¹⁰⁷。这样 Master Orchestrator 默认每轮选取最高优先的任务执行 ¹⁷，保证重要任务（测试开发）不被次要任务（bug修复）饿死。同时，由于测试任务会被Bug阻塞，需要Bug完成后才能继续，所以整体流程仍有序。
- Agent 在执行过程中必须及时更新任务状态。例如，当开始处理任务时使用 `bd update <id> --status in_progress` 标记正在处理 ¹⁰⁸ ¹⁰⁹；当任务完成时用 `bd close <id> --reason "..."` 标记完成 ²⁵ ²⁹；如果任务因异常终止则标记为 `error`。这些状态更新在 Agent SOP 中已有明确步骤要求 ¹¹⁰ ¹¹¹，务必确保Agent严格执行，以便 Orchestrator 可以据此决策下一步（例如跳过处理中任务、识别错误任务等）。
- 定期调用 `bd sync` 保持本地数据库与JSON日志文件同步 ¹¹²。尤其在Agent完成或Orchestrator每轮循环末尾，都应 `sync`（本方案中Orchestrator每处理完一个任务都会执行一次`bd sync`并等待几秒 ¹¹³ ¹¹⁴）。这样可将任务进展记录保存，并触发Git同步（若设置了Git后端），防止长时间运行中数据丢失或不同步。

2. 利用依赖（阻塞）关系管理任务链：

Beads 提供了简洁的依赖管理，通过 `blocks` 类型将一个任务标记阻塞另一个任务。例如：

- 当 Test Creator 创建 Bug 任务时，立即执行 `bd dep add <测试任务ID> <新Bug任务ID> --type blocks` 来建立依赖，并将测试任务状态设为 `blocked` ²⁴ ¹¹⁵。此设计意味着测试任务需等待Bug任务完成才可继续。
- 一旦 Bug 任务被关闭（`closed`），Beads 自动解除对测试任务的阻塞，使其状态回到 `ready`（若无其他阻塞）。Orchestrator 下次循环取到该测试任务时，即可让 Test Creator 继续未完成的流程。
- 同样道理，可以建立更复杂的依赖链。例如一个任务可能依赖多个前置任务，都完成才会变为 `ready`。这种场景可用 `bd dep add` 多次添加阻塞关系实现。Master Orchestrator 会在**所有依赖解除**后才获取到该任务。善用这一机制可以串联多步骤流程而不需要写额外的代码逻辑。

要注意避免循环依赖或永久阻塞的情况发生。前述 Orchestrator 中的死锁检测就是为了捕获异常的依赖关系（例如任务A阻塞任务B，任务B又阻塞任务A的错误配置）⁴⁹ ⁵⁰。因此在创建依赖时，应确保依赖方向正确且最终可达成完成条件。

3. 合理拆分任务粒度：每个 Beads 任务应尽量对应一个明确且独立的工作单元。避免将过大的目标放在一个任务里，以致无法准确标记状态或阻塞。例如，测试开发和结果验证宜拆分成独立任务（Test任务和Validation任务），通过前述方式串联，而不是在一个任务内全部完成。这样不仅方便分配给不同Agent处理，也使任务状态更具意义（测试任务完成表示代码已跑通，验证任务完成表示结果已核实）。

4. 批量或长期任务管理：若有批量的测试任务（比如几十个测试用例需要生成），可利用 Beads 的批处理能力：将任务导入 JSONL 或通过脚本批量创建多个任务。Orchestrator 会按优先级顺序逐一执行，可以通过设置不同优先级或合理安排依赖来控制执行次序。对于长期运行的Master实例，建议定期使用 `bd stats` 或 `bd list` 检查任务队列健康状态，及时清理可能的遗留任务（例如已无关紧要的blocked任务）。

5. 结合CI/CD与通知：当集成 CI/CD 时，可以让流水线或Hook使用 `bd` 命令与任务系统交互。例如在代码库有更新时自动创建相应测试任务，或在任务全部完成时触发 `bd export` 生成报告。在通知方面，可利用 `bd` 提供的状态，定时将任务状态汇总发送到Slack或生成Dashboard，使团队了解测试进展。这部分需要编写额外脚本查询 Beads 数据并输出，项目结构已预留 `integrations/` 目录用于此类脚本放置。

通过以上建议，Beads 将不仅仅是一个被动的任务记录工具，而真正成为多Agent系统的协调中枢：所有任务事件通过它来持久化和传递，各Agent通过它实现解耦协作。良好的 Beads 使用规范，确保了任务在正确的时间被正确的Agent处理，并为系统提供了跨会话、跨团队的信息共享机制。例如，在多人协作时，一名成员创建的任务也会出现在 Beads 列表中，Master即可调度Agent处理，实现半自动的团队协作测试¹¹⁶ ¹¹⁷。

值得一提的是，Beads 数据以 SQLite 和JSONL双格式存储，天生支持Git版本控制⁹。这意味着所有任务和过程记录都可被审查和追溯。当测试发现新的问题、Agent 采取了何种行动，这些历史都可以从版本库中还原。这为改进测试流程、培训AI Agent以及满足合规审计都提供了宝贵的数据依据。

结论

通过本次优化，E2E-Automation-Optimized 项目的架构和实现更趋成熟与完善：

- **架构层面**，引入了清晰的分层设计和模块边界，**知识库-工作流程-任务管理-执行**各司其职又紧密配合，奠定了系统稳定运行和日后扩展的基础。
- **调度层面**，重新设计的 Master Orchestrator 逻辑清晰健壮，结合严格的任务命名规范，实现了**多Agent事件驱动协作**。Agent 之间通过任务和依赖解耦衔接，极大提高了流程自动化程度和容错性，并可平滑拓展并发能力。
- **流程编排层面**，SOP 文档的结构化重构和模板化提升了流程标准化水准，使新增流程或变更现有流程更加轻松可靠。AI Agent遵循标准SOP行动，行为可预测可控，同时SOP自身变得易读易维护。
- **项目组织层面**，调整后的目录结构模块清晰、层次分明，并为**CI/CD、性能测试、团队协作**等未来需求预留了接口和位置。无论项目本身迭代，还是推广到其他项目（横向扩展更多测试项目），都能沿用这套结构，避免推倒重来。
- **工具使用层面**，充分发挥了 Kiro CLI 与 Beads 的威力。Kiro CLI 让每个Agent各尽所能，而Beads串联起所有Agent，提供了一个**持久且共享的记忆**。借助这些工具的优化使用，系统实现了从“人工驱动测试”向“完全自主测试”的飞跃¹¹⁸ ¹¹⁹。

总而言之，优化后的 E2E-Automation-Optimized 系统已具备生产级的健壮性和可扩展性。从单Agent模式无缝升级到多Agent自治协同，它证明了 AI Agent 在测试领域落地的巨大潜力。展望未来，我们计划引入**并行执行、多角色扩充、CI/CD 深度融合**等功能 ¹²⁰ ¹²¹，进一步增强系统能力，最终迈向一个**团队级部署的完全自主测试系统** ¹²²。相信随着这些优化的实施，E2E 自动化测试将更高效可靠地护航产品质量。

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

39

40

41

42

43

44

45

46

47

48

49

50

51

52

53

54

55

56

57

58

59

60

61

62

63

64

65

66

67

68

69

70

71

72

73

74

75

76

77

78

79

80

81

82

83

84

85

86

87

88

89

90

91

92

93

94

95

96

97

98

99

100

101

102

103

104

105

106

107

108

109

110

111

112

113

114

115

116

117

118

119

120

121

122

projectsummary.md

file:///file_00000000e8c471fd9988c75f75025564